

The solutions for this sheet must be submitted on Moodle by 07 May, 15:59.

THE FOLLOWING EXERCISE IS WORTH 2 BONUS POINTS.

Exercise T5.1 – Sum of Unit Vectors

- (a) By $\|v\| := \sqrt{v \cdot v}$ we denote the Euclidean norm in $\mathbb{R}^n$. Let $v_1, \dots, v_n \in \mathbb{R}^n$ be unit vectors. That is, $\|v_i\| = 1$ for all $1 \leq i \leq n$. We consider the Laplace space $\Omega = \{-1, 1\}^n$, and the random variable $X : \Omega \rightarrow \mathbb{R}_{\geq 0}$ defined by

$$X(s_1, \dots, s_n) := \|s_1 v_1 + \dots + s_n v_n\|^2.$$

Note that

$$X(s_1, \dots, s_n) = \sum_{i=1}^n \sum_{j=1}^n s_i s_j \cdot (v_i \cdot v_j),$$

where $v_i \cdot v_j$ denotes the scalar product of v_i and v_j . Show that $\mathbb{E}[X] = n$.

- (b) What does this imply on the minimal possible norm of a vector v obtained by choosing $s_1, \dots, s_n \in \{-1, 1\}$ and setting $v := s_1 v_1 + \dots + s_n v_n$?
- (c) In the setting of (a), give an upper bound on the probability $\Pr[\|s_1 v_1 + \dots + s_n v_n\| \geq 2\sqrt{n}]$.
- (d) Design a Las Vegas algorithm which takes as input unit vectors $v_1, \dots, v_n$ and outputs signs $s_1, \dots, s_n \in \{-1, +1\}$ with $\|s_1 v_1 + \dots + s_n v_n\| \leq 2\sqrt{n}$. What is the expected running time of your algorithm (in O -notation)?

THE FOLLOWING EXERCISE DOES NOT GIVE BONUS POINTS.

Exercise T5.2 – Hash tables

In the lecture you have already seen how to find all duplicates in a data set. In this exercise, two things are different. First, we assume that the elements in our data set can be read and compared in constant time (if you want to, you may also assume that they can be copied in constant time). Second, we only want to determine if there are any duplicates or not: We are given as input an array of numbers $a_1, a_2, \dots, a_n$, say of type `double`. The problem is to produce either a pair of indices $1 \leq i < j \leq n$ such that $a_i = a_j$ or, if no such pair exists, output “ALL ENTRIES ARE DISTINCT!”.

Deterministically, this problem can be solved in $O(n^2)$ using two for-loops. A slightly more clever approach using fast sorting algorithms solves this in $O(n \log n)$ time. In practice, however, an even faster way to solve this problem is using hash tables.

Suppose we are given a hash function h that takes as input a `double` and outputs an integer between 1 and n . The idea is to first construct n lists $L_1, L_2, \dots, L_n$ where L_k contains all indices $i \in \{1, 2, \dots, n\}$ where $h(a_i) = k$, and then investigate the content of each list in more detail.

- (a) Let n , the sequence $a_1, \dots, a_n$, and a hash function h be given. Let $C = \{(i, j) : a_i \neq a_j \text{ and } h(a_i) = h(a_j)\}$. Give an algorithm that solves the element uniqueness problem in $O(n + |C|)$ time and $O(n)$ memory.

We assume here that $h(x)$ for any number x can be computed in $O(1)$ time.

- (b) Suppose h is randomly chosen in such a way that for all $a \neq b$ we have

$$\Pr[h(a) = h(b)] = \frac{1}{n}.$$

Show that, in this case, the expected running time of your algorithm is $O(n)$.